Lesson 6:
RAG Evaluation & Continuous Integration

Shreya Shankar & Hamel Husain

Last time / today

Last time

- Built an LLM judge for one failure mode
- Wrote a judge prompt with pass/fail rules and examples
- Split labels into train, dev, and test
- "Hill-climbed" on the dev set to improve TPR and TNR
  - Inspected disagreements, revised pass/fail boundary
  - Added examples for action confirmations and product listings
- End result: a "frozen" prompt and model for one LLM judge

Today

- Evaluating the retrieval pipeline
- Writing eval cases from failure modes
- Handling nondeterminism with passk and pass@k
- Continuous integration with GitHub Actions

Whiteboard: evaluating retrieval

Two-stage retrieval pipeline and per-stage metrics

Our failure taxonomy from lecture 4

Failure mode	Count	Eval?
Reply includes fields or policy the user did not ask about	69%	✔ LLM judge
Reply exposes internal document ID (e.g., cw-refunds)	38%	✔ code check
Refund or cancellation executed without user confirmation	18%	✔ code check
Reply says refund is complete but tool returned queued	12%	✔ code check
Reply contradicts the tool result (e.g., wrong amount)	9%	✔ code check
Redundant policy lookups before answering	15%	✘ fix the prompt
Forgot budget or preference from earlier turn	6%	✘ fix the prompt
Ceramics store returned for a jam query	3%	✘ fix retrieval
Product listing without photos	3%	✘ fix the prompt
"I completely understand your frustration" on a neutral question	3%	✘ fix the prompt

Components of an eval

- **Tasks**: seed each task from a failing trace
  - Each task has an input, an initial database state, and a rubric
  - Around 5 tasks per failure mode
  - **Regression task:** agent already passes, guard against breakage
  - **Capability task:** agent sometimes fails, track improvement over time
- **Rubric**: how you check whether the agent passed
  - Programmatic verifier, e.g., string match, database query, tool call check
  - LLM judge

Tasks in Harbor format

Harbor: an open source framework for defining and running agent evals. docs.harborframework.com

```
eval_cases/
  e-001-unconfirmed-write/
    task.toml
    instruction.md
    test.py
  e-002-internal-doc-cited/
    task.toml
    instruction.md
    test.py
  e-003-verbose-reply/
    task.toml
    instruction.md
    test.py
  ...
```

- Each eval case is a directory with three files
- **task.toml**: metadata and initial database state
- **instruction.md**: the user message the agent receives
- **test.py**: the rubric that checks the result
- Run all tasks: `harbor run -p eval_cases/`

Walkthrough: verbose reply eval case

From lecture 4: the agent dumped every order field when the shopper only asked if an order shipped.

```
# task.toml

[task]
id = "e-003"
failure_mode = "verbose_reply"
type = "capability"

[input]
role = "shopper"
user_id = 1

[initial_state]
seed = "default"
order_id = 335
```

failure_mode = "verbose_reply"
The failure mode we found in lecture 4 and built a judge for in lecture 5

type = "capability"
Agent sometimes passes, sometimes fails. Track the pass rate instead of blocking on every failure

order_id = 335
Seed the database so order 335 exists with status "shipped" before the agent runs

Walkthrough: instruction and rubric

instruction.md

```
I ordered a Bluetooth speaker
from Cascade Audio a little
while ago and I'm wondering
if it's been shipped yet?
```

test.py

```
def test(trace, db):
    result = run_judge(
        "verbose_reply_v2",
        trace
    )
    assert result["passes_mode"],
        result["evidence"]
```

Walkthrough: instruction and rubric

instruction.md

```
I ordered a Bluetooth speaker
from Cascade Audio a little
while ago and I'm wondering
if it's been shipped yet?
```

test.py

```
def test(trace, db):
    result = run_judge(
        "verbose_reply_v2",
        trace
    )
    assert result["passes_mode"],
        result["evidence"]
```

What run_judge does

```
## Task
Decide whether the last reply gives the
user the information needed, without
unnecessary detail.

## Fail rules
- Dumps order fields the user did not ask about
- Exposes internal system state
- Cites internal doc IDs like (cw-refunds)
- Compares platform and store policy when
  only one applies

## Output
Write a critique, then "Pass" or "Fail".
```

Simplified for the slide. The real prompt has examples, boundary cases, and an output format.

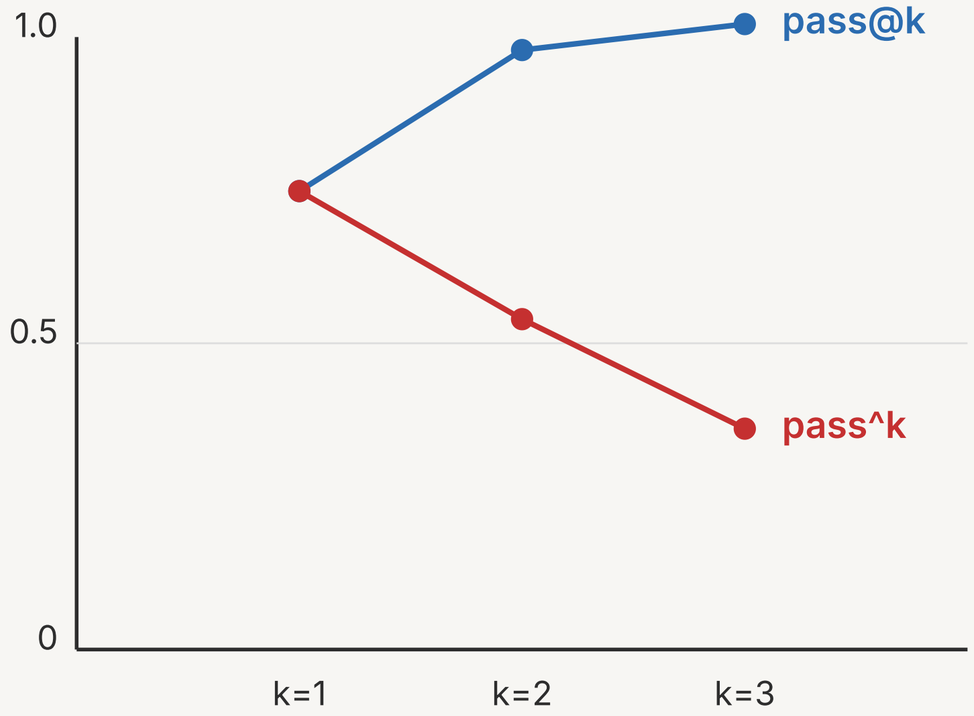
High TPR is critical (otherwise false alarms block every merge)

Metrics for running evals

- The agent is nondeterministic: the same input can pass on one run and fail on the next
- **pass@k**: probability that at least one of k runs passes
  - `pass@k = 1 - C(n-c, k) / C(n, k)` (n runs observed, c pass)
  - For capability tasks: the agent can handle the case, but not reliably yet
- **passk**: probability that all k runs pass
  - `pass^k = C(c, k) / C(n, k)`
  - For regression tasks: the agent must handle the case every time

Example: tau-bench

Yao et al., 2024



6 of 8 runs pass

tau-bench, airline domain

	pass1	pass2	pass3	pass4
Claude 3.5 Sonnet	0.46	0.33	0.26	0.23
GPT-4o	0.37	0.24	0.18	0.14

Even the best models drop from 46% to 23% when every run must pass

How to think about evals

Rows: failure modes from axial coding. Columns: tasks (concrete requests). The goal is to expand the matrix.

	"Has my speaker shipped?"	"Cancel my last order"	"What's the return policy for Northwind Books?"	"Refund my broken vase"	…
Verbose reply	✔	✔	○	○	←
Internal doc ID exposed	○	○	✔	○	←
Unconfirmed write	○	✔	○	✔	←
Misreported tool state	○	○	○	✔	←
…	↑	↑	↑	↑	

- ✔ = eval case exists ○ = not yet covered
- Start from the traces you annotated, then add tasks to fill the gaps
- Around 5 tasks per failure mode, ~30 tasks total for CI

Continuous integration

- A suite of ~30 eval tasks that run on every pull request
- Mix of regression tasks (passk) and capability tasks (pass@k)
- Cover the failure modes discovered in axial coding
- Continuously add new tasks and retire old ones as the agent changes
- **The suite should never be 100% passing**

- Run locally with `harbor run -p eval_cases/`
- Run in CI with GitHub Actions (on every PR)
- Each run starts from a clean database state (Modal sandboxes)

Homework 6 and upcoming

- Create at least one regression case and one capability case
- Run five baseline trials per case using Harbor on Modal
- Compare pass@k after 5, 10, and 15 trials

When	What
Tue 9/22, 11:00 AM PT	Guest: Getting started with simulations (Alexis Gauba, Ben H.)
Tue 9/22, 1:00 PM PT	Guest: Evaluating document AI (OCR, agentic)
Wed 9/23, 9:30 AM PT	Office hours
Thu 9/24, 2:00 AM PT	Study group (APAC friendly)
Thu 9/24, 10:00 AM PT	Study group
Sat 9/26, 11:00 AM PT	Lecture 7: Safety and adversarial evaluation
Sat 9/26, 12:15 PM PT	Office hours

Homework 6 instructions

Course reader